#4 We compute the distance between current node's point and sphere's center.
#5 If that's less than the sphere's radius, we can add current point to the results.
#6/#7 Check which branch is on the same side of the sphere's center (close) and which on the other side (far).
#8 We certainly need to traverse the closest branch, because it intersects the sphere. Add all point found to the results for current subtree.
#9 Using one of the helper functions defined in listing 9.2, we compute the distance between the split line passing through current node and the center of the sphere. If this distance is closer than the radius, then the furthest branch will also intersect the sphere (see figure 9.25).
#10 Traverse furthest branch, and add all points found to current results.
#11 Return the points found in this subtree.

The other region we are going to use for these queries is a rectangular shape. As you can imagine, the only difference with respect to the other method is the way we check if we have to prune a branch. If we assume that the rectangle is oriented along the cartesian axes used for splits, then pruning might even be considered easier, as shown in figure 9.26: suppose we are at a horizontal split, then we need to understand if the split line intersects the search region, and in this case traverse both branches, or otherwise if it's above or below the region, which will tell us which branch we can prune. This can be checked by simply comparing the y coordinate current node's point, call it `Ny`, with top (`Rt`) and bottom (`Rb`) y coordinates of the rectangular region, as we can have three cases: